

Behavioral Synthesis of ICs

Architectural Models in Synthesis

Credentials: majority of information are from the textbook:

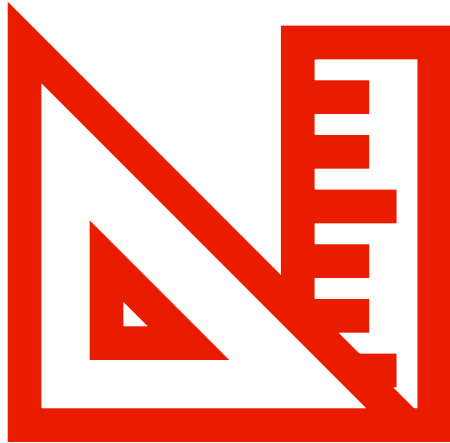
Gajski, Daniel D., Nikil D. Dutt, Allen CH Wu, and Steve YL Lin. *High—Level Synthesis: Introduction to Chip and System Design*. Springer Science & Business Media, 2012.

Outline

1. Design Styles and Target Architectures.
2. Combinatorial Logic
3. Finite State Machines
 - Autonomous FSMs .
 - State-Based and Transition-Based FSM
4. Finite State Machine with a Datapath
5. System Architecture
6. Engineering Considerations
 - Clocking.
 - Busing
 - Pipelining

Objectives

1. To introduce the concepts of design styles and target architectures.
2. To review the concepts of combinatorial logic and finite state machines.
3. To introduce the concepts of autonomous finite state machines, state-based and transition-based finite state machines, and finite state machines with a datapath.
4. To introduce the concepts of system architecture and engineering considerations, including clocking, busing, and pipelining.



Design Styles and Target Architectures

In the context of hardware and integrated circuit design, the design style could refer to the way circuits are constructed

Design Styles and Target Architectures

- During the **refinement** of a given specification into a **structural design** of standard components, a designer first selects a **design style** and then defines a **target architecture**.
- The term **design style** refers to the **principal qualitative features of a design**, such as prioritized interrupt, instruction buffer, snooping data cache, bus-oriented datapath, serial I/O, direct memory access, and others.
- A **target architecture** defines a design more precisely in terms of particular units, their parameters, and the connections among units.
- For **example**, a processor architecture would include the number of registers in the register file, the number of buses in the datapath, the number of pipeline stages, the number of status bits, the number of ways branching can occur, and so on.

Design Styles and Target Architectures

- Each synthesis program or algorithm assumes a certain target architecture.
- When the target architecture is more realistic, the quality of design generated at the end of the synthesis process is higher.
- More realistic target architectures, however, require more sophisticated algorithms, so there is an obvious tradeoff between design quality and the complexity of synthesis algorithms.
- **For instance**, consider the 3-bus architecture in next Figure 1.a:
 - Two operands can be fetched concurrently from the register file (RF), operated upon in the arithmetic logic unit (ALU), and the result stored back in the RF in one clock cycle.
 - If variables a , b , c , d , x , and y are in arbitrary registers in the RF and each cycle takes 100 ns, then the two following binary operations can be executed in two clock cycles or 200 ns:

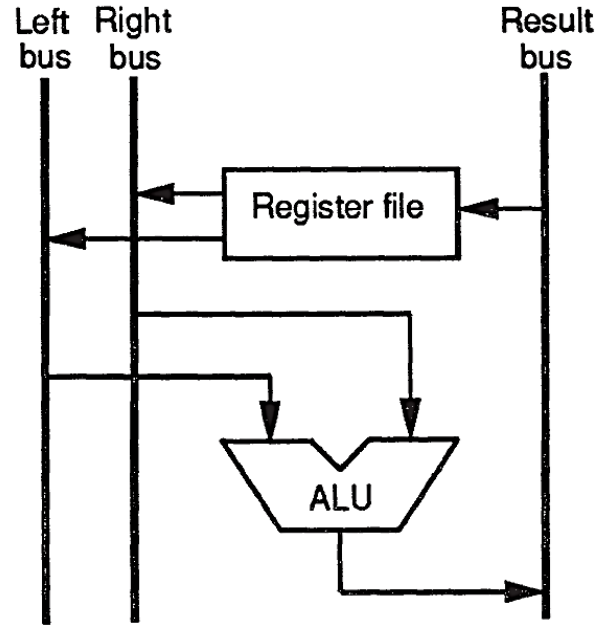
$$1. \quad x \leftarrow a + b \quad (100 \text{ ns})$$

$$2. \quad y \leftarrow c - d \quad (100 \text{ ns})$$

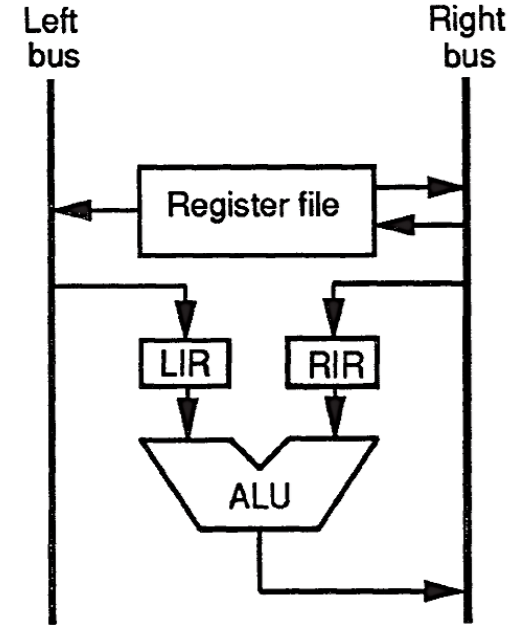
Design Styles and Target Architectures

→ **Figure 1:** Two different datapath styles:

- a) 3-bus nonpipelined design,
- b) 2-bus pipelined design.



(a)



(b)

Design Styles and Target Architectures

- The datapath model shown in the previous Figure 1.a is oversimplified and unrealistic.
- Since buses consume a large portion of a **chip's area**, three buses may not be the most efficient use of silicon.
- In addition, the RF and the ALU are used only during **a portion of the clock cycle**, and, more importantly, they are used exclusively. When signal changes are propagated through the ALU, no signal changes are propagated through the RF and vice versa.
- A significant performance gain can be obtained by inserting two registers, LIR and RIR, before the inputs of the ALU, by halving the clock cycle, and by increasing the usage of the RF and the ALU in each clock cycle.
- A datapath model reflecting this architecture is shown in Figure 1.b.

Design Styles and Target Architectures

→ Within this model, the earlier two operations can be executed at the same time as:

1. $LIR \leftarrow a; RIR \leftarrow b;$ (50 ns)

2. $x \leftarrow LIR + RIR; LIR \leftarrow c; RIR \leftarrow d;$ (50 ns)

3. $y \leftarrow LIR - RIR$ (50 ns)

→ and can be executed in three clock cycles or 150 ns, resulting in a 25% improvement over the previous architecture.

→ **Note**, however, that we obtained this improvement because the two binary operations were **independent** of one another.

→ **Data dependency** in computing refers to a situation where a program instruction is dependent on the data result of a previous instruction. This means that the current instruction cannot proceed until the needed data, produced by the prior instruction, is available.

Design Styles and Target Architectures

→ For **example**, there would be no gain if we had to execute the operation $x \leftarrow a + b + c$. It would require two clock cycles on the first architecture:

1. $temp \leftarrow a + b;$ (100 ns)

2. $x \leftarrow temp + c;$ (100 ns)

→ and four clock cycles on the second:

1. $LIR \leftarrow a; RIR \leftarrow b;$ (50 ns)

2. $temp \leftarrow LIR + RIR;$ (50 ns)

3. $LIR \leftarrow temp; RIR \leftarrow c;$ (50 ns)

4. $x \leftarrow LIR + RIR$ (50 ns)

→ In other words, it would execute in 200 ns in either case.

→ No gain is possible since the second operation is **dependent** on the result of the first operation.

Design Styles and Target Architectures

- The architectural model depicted in the last Figure (a) is simple and requires only a simple **scheduling algorithm** for assigning operations to clock cycles.
- The model in Figure (b) is more realistic but requires a synthesis procedure that must include both a construction of the **dependence** relations among operations and a scheduling algorithm that attempts to improve performance by reordering operation execution according to these dependencies.
- This example shows that an oversimplified model of the target design may require a simple synthesis algorithm, but it may generate an inferior design unacceptable to real designers.
- We will describe the basic target architectural models used in logic, register-transfer, and system designs.
- We will also briefly describe the design process used to generate these architectures from input descriptions.

Combinatorial Logic

Combinatorial logic (also known as combinatorial circuits or combinational logic) is a concept in digital circuit design. Unlike sequential logic circuits, where the output depends on the current input as well as the history of inputs (previous states), combinatorial logic's output is only a function of the current input values.

Combinatorial Logic

- **Microarchitectural** components are composed of **functional units** and **storage units**.
 - **Functional units** perform transformations on data values. Basic functional units include adders, comparators, ALU s, multipliers, shifters, incrementors, and selectors, ...
 - **Storage units** preserve those values over time. Basic storage units include registers, register files, and memories, ...
- Functional units can be further divided into **unsliceable logic** (such as encoders, decoders and carry look-ahead generators), **sliceable logic** (such as logic units, selectors, and shifters), and **mixed units** that can be implemented both as sliceable and unsliceable logic (such as adders, subtractors, parity generators, and comparators).
- For **example**, an **adder** can be implemented as a sliceable component using the **ripple-carry** style, or as an unsliceable unit using a **carry lookahead** style.

Combinatorial Logic

- All functional units are **blocks of combinatorial logic** whose outputs can be described by **Boolean expressions** of their inputs.
- For **example**, a 16-bit adder with data inputs $A = \{a_i \mid 0 \leq i \leq 15\}$, $B = \{b_i \mid 0 \leq i \leq 15\}$, and input carry C_{in} , and data output $SUM = \{sum_i \mid 0 \leq i \leq 15\}$ and output carry C_{out} , can be described by the loop:

$$\begin{aligned}c_0 &= C_{in} \\ c_{i+1} &= (a_i \text{ AND } b_i) \text{ OR } (c_i \text{ AND } (a_i \text{ OR } b_i)) \\ sum_i &= a_i \text{ EXOR } b_i \text{ EXOR } c_i \\ C_{out} &= c_{16}\end{aligned}$$

- where the carries c_i are defined with a recurrence in which each carry bit c_{i+1} depends on the previous carry c_i .
- These equations describe a ripple-carry style adder.
- By a backward substitution, we can make each carry bit c_i independent of all carry bits c_j , for $j \leq i-1$. Our new equations roughly describe a carry look-ahead style adder.
- Although the carry look-ahead style speeds up carry propagation, it makes the computation of each carry bit nonuniform, since each carry bit c_i depends on all inputs a_j and b_j , for $j < i$. Consequently, the implementation of a carry lookahead adder is unsliceable and, as a result, difficult to layout on silicon.

Combinatorial Logic

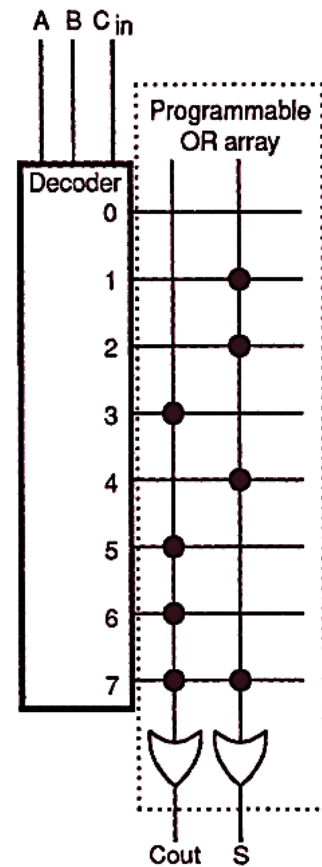
- Another format for **describing a functional unit is with a table** that designates output values for each combination of input values.
- Obviously, this form of description is **limited** to functional units with a **small number of inputs**.
- The tabular form can be extended to functional units with a larger number of inputs if it has output values of 0(1) for a very small number of its input values. In this case, we can tabulate only the input combinations for those output values of 0(1).
- Functional units can be implemented in several different ways. One simple implementation uses a read-only memory (**ROM**), as shown in Figure 2.a. The output value is programmed into a ROM for each combination of input values.
- The function programmed into the ROM in Figure 2.a outputs the binary representation of the number of inputs equal to 1. For three inputs A, B, and C_{in} , this unit, generating sum and carry outputs S and C_{out} , is called a full adder.

Combinatorial Logic

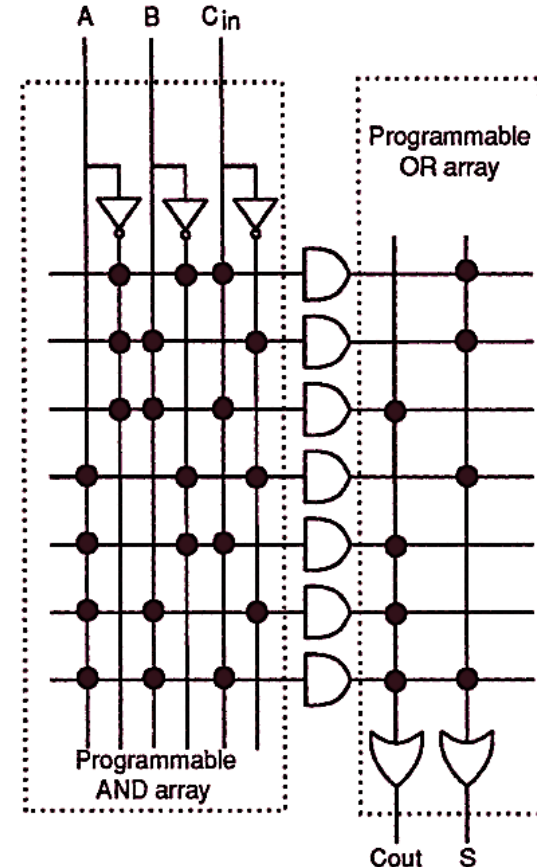
→ **Figure 2:** Full adder implementation with:

- a) a ROM,
- b) a PLA.

→ **Note:** The existence of a dark dot in the OR array indicates a connection between the input decoder and an OR gate whose output is equal to 1 when a proper combination of input values is decoded.



(a)



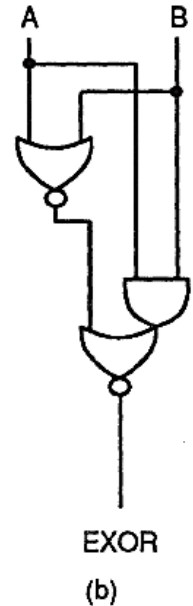
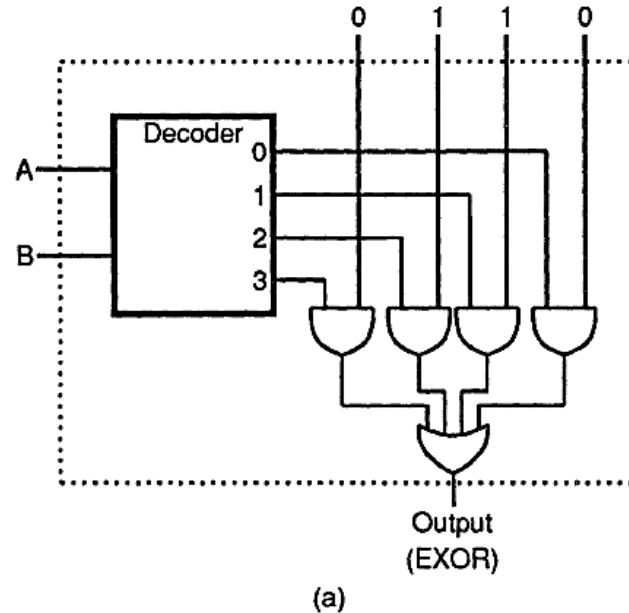
(b)

Combinatorial Logic

- Combinatorial functions also can be implemented with a **programmable logic array (PLA)**, as shown in Figure 2.b.
- Whereas a ROM architecture requires a full decoder, a PLA architecture allows a designer to specify only those input combinations in the programmable AND array that make outputs equal to 1(0).
- In the case of the full adder, only seven of the eight input combinations must be decoded.
- **Small combinatorial functions** can be implemented using a selector (multiplexer) circuit, as shown in Figure 3.a. A selector can be thought of as a single-output dynamically programmable ROM.
- The control inputs correspond to ROM inputs, and the data inputs serve the role of connections in the OR array. The 4-input selector in Figure 3.a implements the well-known exclusive-OR (EXOR) function of two variables, A and B.
- Obviously, the same selector can implement any combinatorial function of two variables.

Combinatorial Logic

→ **Figure 3:** EXOR implementation with: (a) a selector, (b) logic gates.



Combinatorial Logic

- Figure 3.b shows the EXOR circuit implemented using a 2-input NOR gate and a 2-input AND-OR-INVERT gates.
- Comparing these different implementations, we can see that a ROM has a full decoder, while a PLA has a programmable AND array.
- A ROM implementation requires more silicon area than a PLA implementation for functions whose output is 0(1) for only a few input combinations.
- Adding an extra input to ROM doubles the silicon area, while adding an extra input to a PLA increases the area only slightly.
- Similarly, the selector implementation takes more silicon area than logic gates. For that reason, the selector implementation is rarely used.

Combinatorial Logic

- With regard to timing, ROMs, PLAs, and selectors provide two-level logic implementations of combinatorial functions.
- When the number of variables increases, so does the number of inputs to the AND and OR gates.
- A large number of inputs results in **long charging** and **discharging times**, which translates into **long delays**.
- In addition, the regular structure of ROM, PLA and selector circuits introduce **long wire connections**, increasing the **load capacitance** and further encumbering circuit **delay**.
- In contrast, **Boolean gates** allow multi-level implementations using gates with only a few inputs each.
- Despite increasing the levels of gating, the total performance is much better than equivalent ROM, PLA, and selector implementations because shorter wires and fewer transistors in series allow much faster switching times.
- In general, Boolean gates are preferred when performance is the main design goal, but ROMs and PLAs may be preferred for reasons of **manufacturability**, **maintainability**, **verifiability**, and **testability**.

Combinatorial Logic

- The **synthesis process** for combinatorial functions consists of the following tasks (for ROM, PLA and selector implementations, only steps 1 and 2 are needed):
 1. **Compilation**
 2. **Minimization**
 3. **Technology mapping**
 4. **Optimization**
 5. **Transistor sizing**
- A **functional unit** can be described in a variety of languages and in many ways using the same language.
- **Compilation** provides a translation from the input description to a standard intermediate form used by **minimization** and **optimization** algorithms.
- **Minimization** reduces the number of terms and the number of variables in each term, which in turn results in a **smaller number of transistors** needed for implementation.
- Technology mapping replaces Boolean operators with Boolean gates from the given library.

Combinatorial Logic

- Although **minimization reduces the number of operations** in the Boolean expression, it does **not necessarily produce the most efficient implementation**.
- The **efficiency** of the implementation depends on the **library of gates available** to the designer.
- **Optimization** fine-tunes the gate-level implementation by reducing **critical paths**.
- Critical paths through the design can be reduced by **factoring variables** that are not on the critical path.
- **Optimization procedures** work on the principle that maximal factoring produces a maximal level of gating, which minimizes the number of transistors needed for an implementation but introduces longer delays than necessary.

Combinatorial Logic

- **Less factoring** produces **faster**, more **parallel** versions of the same design that require **more transistors**.
- Optimization algorithms **trade off area cost** for improved **delay** and vice versa.
- The final task in the design process defines **proper transistor sizes** in each gate.
- Each **gate** and each **wire** in the implementation represents a certain **capacitance** that has to be charged and discharged when signals switch from 1 to 0 and 0 to 1.
 - **Large transistors** have **more drive capability**, reducing the switching times on the loads that they drive.
 - On the other hand, they represent **larger capacitive loads** for their drivers.
- **Transistor-sizing algorithms** select proper transistor sizes in order to improve overall performance.

Finite State Machines

Finite State Machine (FSM) is a digital logic circuit that transitions between a finite number of predefined states based on the sequence of inputs it receives. It consists of a set of registers that hold the current state, a set of inputs that affect the state, and possibly a set of outputs that may change as the state changes.

Finite State Machines

- A **finite-state machine (FSM)** is the most popular design model in computer science and engineering.
- The model consists of a set of states, a set of transitions between states, and a set of actions associated with states, transitions or both states and transitions. More formally, a FSM is a quintuple

$$\langle S, I, O, f: S \times I \rightarrow S, h: S \times I \rightarrow O \rangle$$

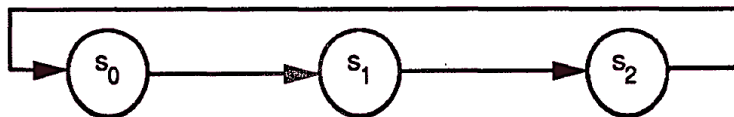
- where $S = \{S_i\}$ is a set of states, $I = \{i_j\}$ is a set of input values, and $O = \{o_k\}$ is a set of output values; f and h are next-state and output functions that map a cross product of S and I into S and O , respectively.
- The basic FSM model can be limited or extended to represent different target architectures.

Autonomous FSMs

- An autonomous FSM is obtained if the input set I is empty. Therefore, next-state and output functions f and h are defined as mappings $S \rightarrow S$ and $S \rightarrow O$.
- This model is used for **free running components** such as the modulo-3 counter shown in Figure 4.
- The function $f: S \rightarrow S$ is shown in Figure 4.a as a **state diagram** in which each node represents a state; transitions between states are represented by unidirectional edges.
- In Figure 4.b, the same state diagram is given in **tabular form**.
 - In this figure, we assign an encoding of two bits to each state.
 - Each bit of information is stored in a storage element such as a D-type flip-flop.
 - The Boolean expressions for the inputs to the flip-flops can be derived from the next-state table.
- In Figure 4.c, an implementation of the FSM is given using two D-type flip-flops, FF_1 and FF_0 . D_1 must be 1 in state s_1 and D_0 must be 1 in state s_0 .

Autonomous FSMs

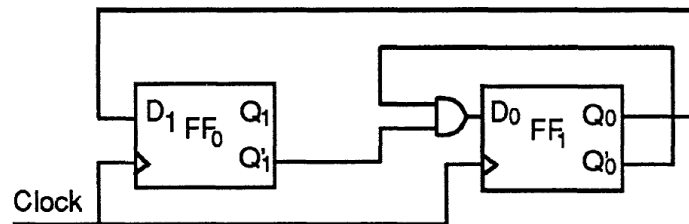
→ **Figure 4:** Modulo-3 counter: (a) state diagram, (b) next-state table, (c) implementation with D-type flip-flops, (d) state waveforms.



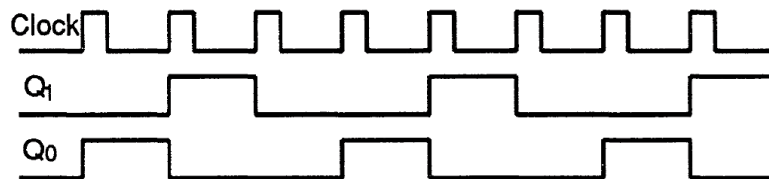
(a)

Present state	Next state
$Q_1 Q_0$	$Q_1 Q_0$
$s_0 = 0\ 0$	$s_1 = 0\ 1$
$s_1 = 0\ 1$	$s_2 = 1\ 0$
$s_2 = 1\ 0$	$s_0 = 0\ 0$

(b)



(c)



(d)

Autonomous FSMs

→ In this circuit:

$$D_1 = s_1 = Q'_1 \text{ AND } Q_0$$

→ Since the state encoding 11 is not used, $Q_1Q_0 = 11$ will never occur. This means that we can rewrite the equation as

$$\begin{aligned} D_1 &= (Q'_1 \text{ AND } Q_0) \text{ OR } (Q_1 \text{ AND } Q_0) \\ &= (Q'_1 \text{ OR } Q_1) \text{ AND } Q_0 \\ &= Q_0 \end{aligned}$$

→ Similarly,

$$D_1 = Q'_1 \text{ AND } Q'_0$$

→ The timing behavior of the modulo-3 counter is shown in Figure 4.d with a waveform diagram. The state is represented by the pair of flip-flop outputs Q_1 and Q_0 .

Autonomous FSMs

- The **example** in Figure 4 demonstrated an FSM without inputs and outputs.
- By adding the output signal Y , which is equal to 1 in state s_2 only, we obtain a modulo-3 divider.
- If Y is ANDed with the clock signal, the result is a clock sequence in which only every third clock pulse appears at the output.
- As another **example**, Figure 5.a shows the next-state table for the modulo-3 divider, and Figure 5.b shows the implementation using D-type flip-flops.
- The Boolean expression for the output Y detects the modulo-3 divider being in state s_2 . Thus,

$$Y = Q_1 \text{ AND } Q'_0$$

- The state and output waveforms for a circuit starting in state s_0 are shown in Figure 5.c.

State-Based and Transition-Based FSM

- Both **state-based** and **transition-based** FSMs have a nonempty input set I .
- They differ in the specification of the output function h .
 - In a state-based (Moore) FSM, the output value depends only on the state of the FSM ($h : S \rightarrow O$),
 - in a transition-based (Mealy) FSM, the output value depends on the state and input values ($h : S \times I \rightarrow O$).
- In a transition-based FSM, the output will change when input value changes, while the state will change only on the next clock pulse.
- In a state-based FSM, the output will persist until the state changes, no matter when the input value changes.

State-Based and Transition-Based FSM

- To illustrate the difference between **state-based** and **transition-based** FSMs, we extended the specification of the modulo-3 divider by adding the control input called *Count*.
 - When *Count* = 1, the modulo-3 divider behaves as previously described.
 - When *Count* = 0, however, the modulo-3 divider returns to state S_0 , in which it persists until *Count* = 1.
- The state-based modulo-3 divider is shown in Figure 6, where the next-state and output functions are described by the tables in Figure 6.a.
- These tables cannot be combined into a single table because the next-state function depends on the input signal *Count* and the present state of the FSM, while the output function depends on the present state, only.
- The implementation using D-type flip-flops is shown in Figure 6.b. The input and output waveforms for an arbitrary input are shown in Figure 6.c.

State-Based and Transition-Based FSM

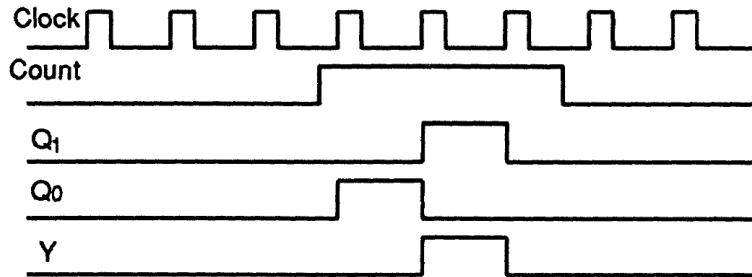
→ **Figure 6:** State-based modulo-3 divider:

- a) next-state and output tables,
- b) implementation with D-type flip-flops,
- c) input and output waveforms.

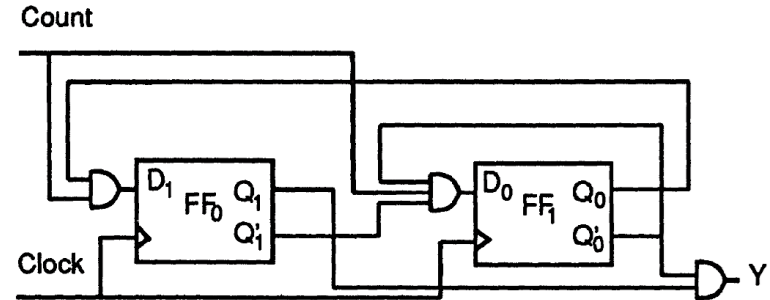
Present state	Input	Next state
$Q_1 Q_0$	Count	$Q_1 Q_0$
$s_0 = 00$	1	$s_0 = 01$
$s_1 = 01$	1	$s_2 = 10$
$s_2 = 10$	1	$s_0 = 00$
don't care	0	$s_0 = 00$

Present state	Output
$Q_1 Q_0$	Y
$s_0 = 00$	0
$s_1 = 01$	0
$s_2 = 10$	1

(a)



(c)



(b)

State-Based and Transition-Based FSM

- In the **state-based** implementation shown in Figure 6, $Y = 1$ during the complete clock cycle in which the modulo-3 divider is in state s_2 .
- Even if *Count* goes to 0 during that clock cycle, Y will retain its output value.
- Such is not the case for the **transition-based** implementation shown in Figure 7.
- **Note** that the output signal Y follows the input signal *Count* when it goes to 0 in state s_2 .

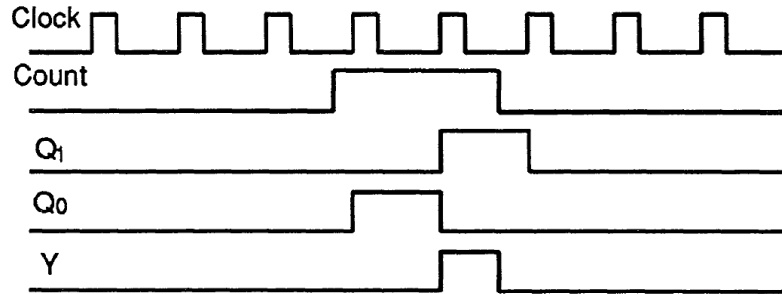
State-Based and Transition-Based FSM

→ **Figure 7:** Transition-based modulo-3 divider:

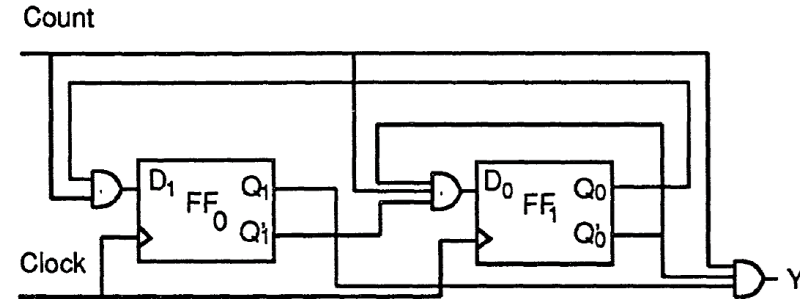
1. next-state and output table,
2. implementation with D-type flip-flops,
3. input and output waveforms.

Present state	Input	Next state	Output
Q_1Q_0	Count	Q_1Q_0	Y
$s_0 = 00$	1	$s_0 = 01$	0
$s_1 = 01$	1	$s_2 = 10$	0
$s_2 = 10$	1	$s_0 = 00$	1
don't care	0	$s_0 = 00$	0

(a)



(c)



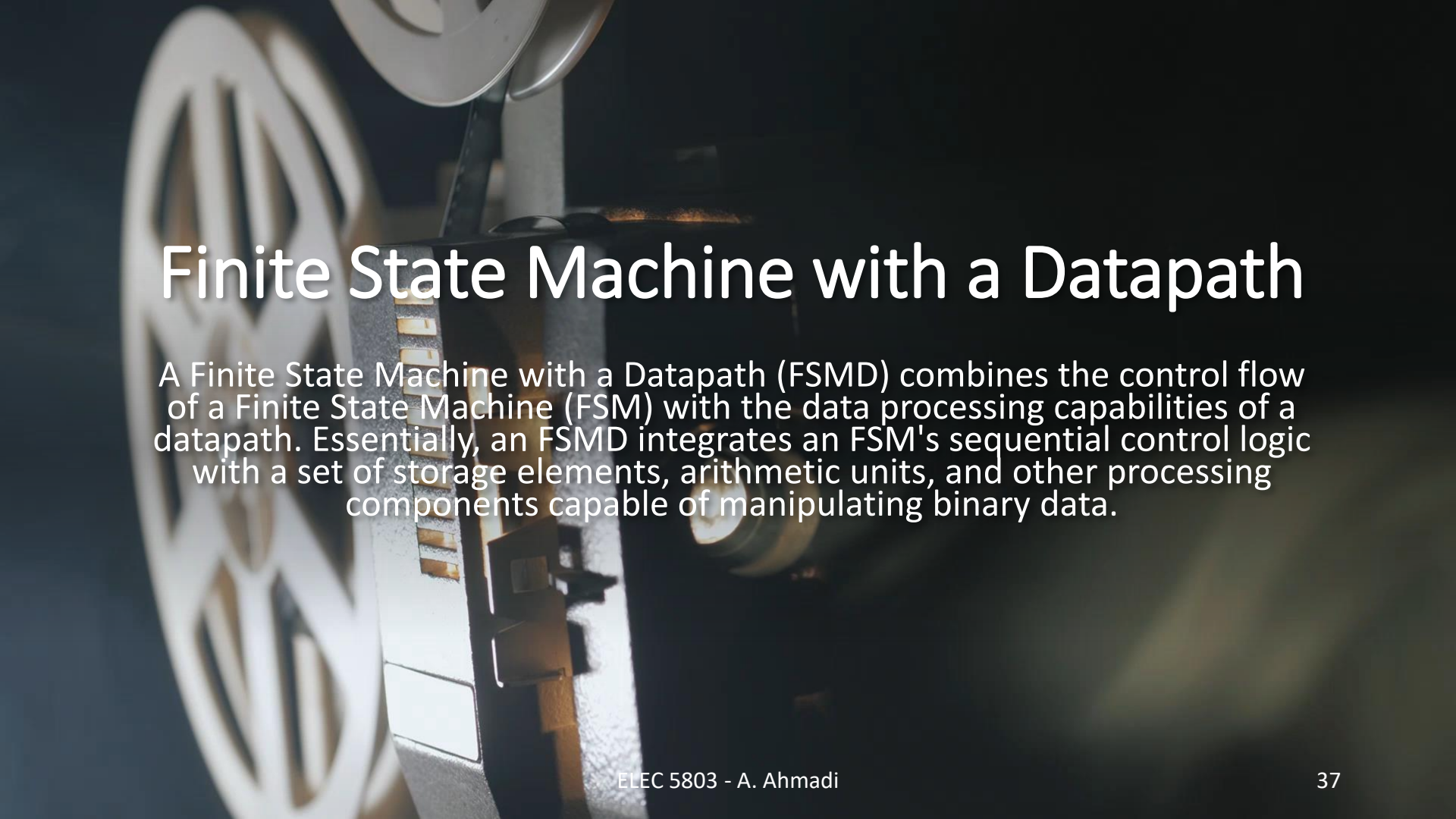
(b)

State-Based and Transition-Based FSM

- The **synthesis process** for state-based and transition-based **FSMs** consists of the following tasks:
 - 1. Compilation**
 - 2. State minimization**
 - 3. State encoding**
 - 4. Synthesis of next-state and output functions**
- **Compilation** translates an input description from a particular language into a standard FSM description.
- The initial description may have **redundant states**. Two states are redundant when every input sequence applied to the FSM in either state generates the same output sequence. In such instances, one of the two redundant states can be eliminated during state minimization.
- After state minimization, each state must be assigned a **binary code**.

State-Based and Transition-Based FSM

- One goal of **state encoding** is the minimization of the number of bits used to encode each state, the result of which is to minimize the number of flip-flops used to store the state encoding.
- The other goal of state encoding is to minimize the implementation of the next-state and output functions.
- This can be done by using a **redundant encoding**: an encoding is redundant if it uses more than the minimal $\log_2(n)$ bits needed to encode n states.
- Although a redundant encoding requires more bits and, thus, more flip-flops, the decoding operations performed by the next-state and output functions can be greatly simplified.
- Thus, the savings in gates for the next-state and output functions may outweigh the additional cost in flip-flops.



Finite State Machine with a Datapath

A Finite State Machine with a Datapath (FSMD) combines the control flow of a Finite State Machine (FSM) with the data processing capabilities of a datapath. Essentially, an FSMD integrates an FSM's sequential control logic with a set of storage elements, arithmetic units, and other processing components capable of manipulating binary data.

Finite State Machine with a Datapath

- The FSM model works well for a few to several hundred states. Beyond several hundred states, the model becomes incomprehensible to human designers.
- Even low-complexity components, such as **I/O interfaces**, and **bus controllers**, can have several thousand states when all the storage elements are counted.
- In order to make the FSM model usable for more complex designs, we introduce a set of **integer** and **floating-point** variables stored in **registers**, **register files** and **memories**.
- Theoretically, each variable replaces thousands of different states.
- For **example**, a 16-bit integer variable represents 2^{16} or 65536 different states; thus, the introduction of a 16-bit variable reduces the number of states in the FSM model by 65536.
- The use of variables leads us to the concept of an **FSM with a datapath** (FSMD).

Finite State Machine with a Datapath

- Let us define a set of storage variables VAR , a set of expressions, $EXP = \{f(x, y, z, \dots) \mid x, y, z, \dots \in VAR\}$, and a set of storage assignments, $A = \{X \leftarrow e \mid X \in VAR, e \in EXP\}$.
- Let us further define a set of status signals as logical relations between two expressions from the set EXP , $STAT = \{Rel(a, b) \mid a, b \in EXP\}$.
- Given these definitions, an **FSMD** can be defined as the quintuple
$$\langle S, I \cup STAT, O \cup A, f, h \rangle$$
- where S , f , and h are defined as before, the set of input values is extended to include a combination of status values, and the output set is extended to include storage variable assignments.

Finite State Machine with a Datapath

- For **example**, a modulo-3 divider can be modeled as a 1-state FSM, as in Figure 8.
- In this implementation, we introduce a 2-bit storage variable x that represents the counter.
 - When x is not equal to 2, x is incremented by 1, and
 - When $x = 2$, x is set to 0.
- The next-state and output functions are shown in Figure 8.a, and the FSM implementation is shown in Figure 8.b.
- The datapath consists of a register for storing variable x , a selector to supply 0 or $x + 1$ to the register, and an adder to increment the value of x .
- The register is loaded with 0 or $x+1$ as long as $Count = 1$. When $Count = 0$, the value of the register is set to 0.

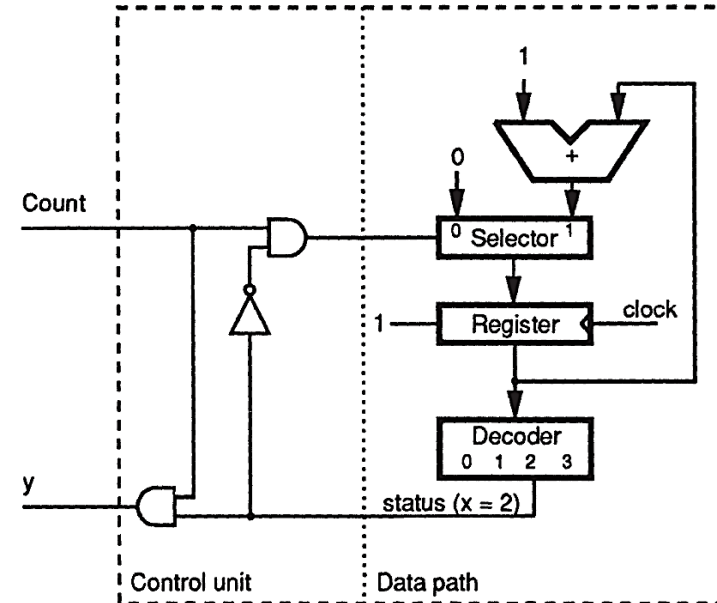
Finite State Machine with a Datapath

→ **Figure 8:** Modulo-3 divider modeled as a transition-based FSMD:

- a) next-state and input tables,
- b) datapath implementation.

Present State	Input	Next State	Output
s_0	(Count = 1) AND ($x \neq 2$)	s_0	$x = x + 1, Y = 0$
	(Count = 1) AND ($x = 2$)		$x = 0, Y = 1$
	Count = 0		$x = 0, Y = 0$

(a)



(b)

Finite State Machine with a Datapath

- A modulo-3 divider also can be modeled as a state-based FSM, as shown in Figure 9, where a different state is used for each distinct combination of output values and storage assignments.
- For this model, we need three different states:
 - s_0 , when $Y = 0$ and $x \leftarrow 0$;
 - s_1 , when $Y = 0$ and $x \leftarrow x + 1$;
 - s_2 , when $Y = 1$ and $x \leftarrow 0$.
- The next-state and output tables for a **state-based architecture** is shown in Figure 9.a, and its implementation is shown in Figure 9.b.
- The **states** are implemented with a 2-bit variable called *State : Exp* and the next-state logic is implemented using two 2-input selectors, a 2-bit decoder, and an AND-OR gate.
- The **output logic** is implemented using one 2- input AND gate. Although the implementation of the next-state and output functions using selectors, decoders, and gates is nonoptimal, it is straightforward to understand and to use for deriving control expressions.

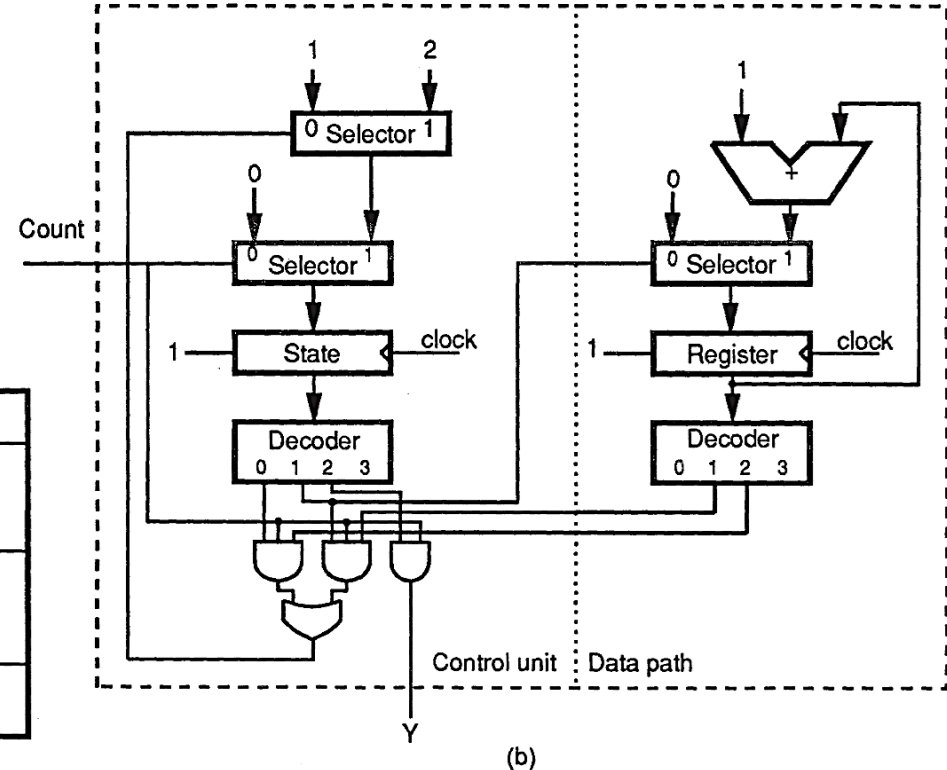
Finite State Machine with a Datapath

→ **Figure 9:** Modulo-3 divider modeled as a state-based FSM:

- a) next-state and output table,
- b) datapath implementation.

Present State	Input	Next State	Output
s_0	Count = 0 (Count = 1) AND (x $\neq$ 2) (Count = 1) AND (x = 2)	s_0 s_1 s_0	x = 0, Y = 0
s_1	Count = 0 (Count = 1) AND (x = 0) (Count = 1) AND (x = 1)	s_0 s_1 s_2	x = x + 1, Y = 0
s_2	Count = 0 Count = 1	s_0 s_1	x = 0, Y = 1

(a)

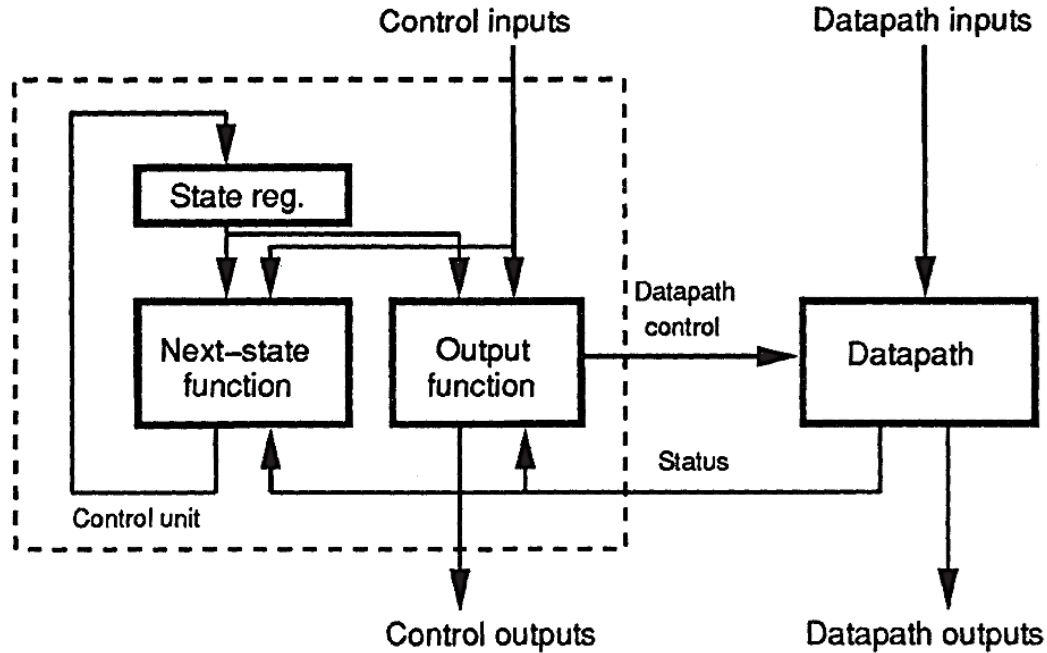


Finite State Machine with a Datapath

- FSMD models are used to describe digital systems on the **register-transfer level**.
- A block diagram of a **generic FSMD** implementation is shown in Figure 10; it consists of an **FSM**, called a **control unit**, and a **datapath**.
 - The **control unit** contains a state register and two combinatorial blocks computing the next-state and output functions.
 - The inputs to the control unit can be categorized as **control inputs** and **status bits**;
 - The outputs can be categorized as **control outputs** and **datapath control signals**.
 - The **datapath** also has inputs and outputs. The datapath inputs and outputs are typically **words**, while control inputs and outputs are more often single **bits**.
- The **FSMD** is state based if the output function does not depend on control inputs and status bits.

Finite State Machine with a Datapath

→ **Figure 10:** Generic FSMD block diagram.



Finite State Machine with a Datapath

- Atypical implementation of the next-state function is presented in Figure 11.
- The **next state** is selected between the incremented value of the present state and a branch state stored in a ROM or PLA.
- The selection is made through the **address** selector by the test bit, which is itself selected through the status selector among control inputs and status bits.
- The **control inputs** represent external conditions, such as **interrupts**, and **status bits** represent **internal conditions**, such as a zero result, overflow, or sign bit.
- The selection code for the test bit is stored with the **branch state** in the ROM or PLA.
- **Note**: the implementation in Figure 11 resembles the control unit of **standard processor** chips.

Finite State Machine with a Datapath

→ The **design process** for FSMDs has some additional tasks in comparison to FSM synthesis. There are eight basic design synthesis steps:

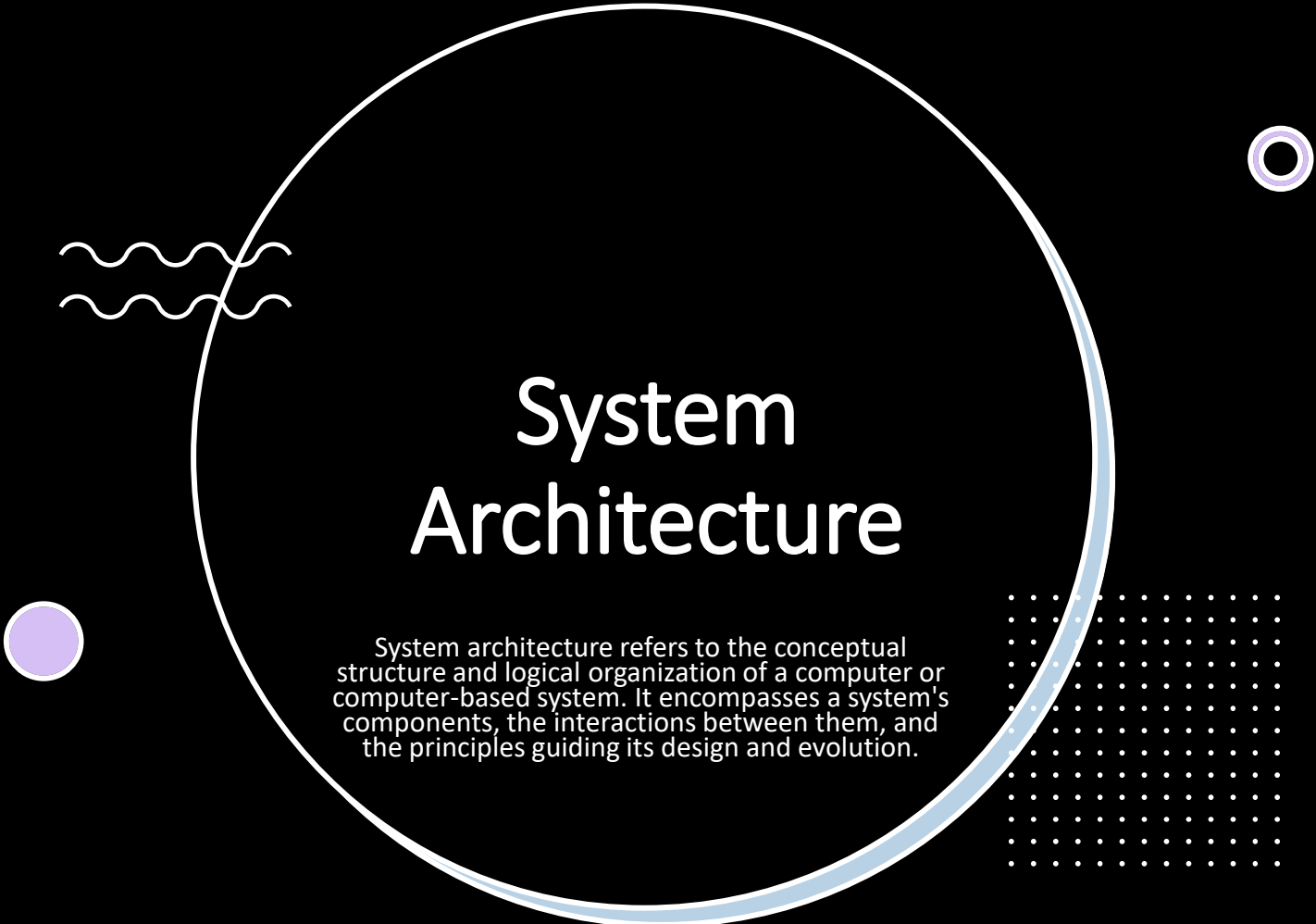
1. Compilation
2. Unit selection
3. Storage binding
4. Unit binding
5. Interconnection binding
6. Control definition
7. Control-unit synthesis
8. Functional-unit synthesis

Finite State Machine with a Datapath

- **Compilation** translates the original description, written in a **hardware description language (HDL)**, into a **tabular form** for the next-state and output functions.
- **Unit selection** determines the number and type of **storage**, **functional** and **interconnection** units to be used in the design.
 - Since the output contains variable assignments, the declared variables are stored in **registers** and **memories**.
 - Variables with **nonoverlapping lifetimes** may share storage elements.
 - Two variables x and y have nonoverlapping lifetimes if variable x is assigned a value whose usage ends before variable y is assigned a value and vice versa.
- **Storage binding** assigns variables to **storage elements**.
 - Similarly, **operations** executed in different states are independent of each other and can be executed by the same **functional unit**.

Finite State Machine with a Datapath

- **Unit binding** assigns **operations** to **functional units**. Selectors or buses are interconnection units that are required to connect storage and functional units.
- **Interconnection binding** assigns interconnection units to datapaths between units.
- Many HLS algorithms perform **selection and binding** together in a process called **allocation**.
- **Control definition** involves generating **Boolean expressions** for every datapath **control signal** that controls selectors, storage units, and functional units.
- **Control-unit synthesis** and **functional-unit synthesis** generate detailed designs for the control unit and datapath using the previously defined FSM and FU design processes.
- All of these eight tasks are interdependent in the sense that optimally solving each will not necessarily guarantee overall optimality of the design.



System Architecture

System architecture refers to the conceptual structure and logical organization of a computer or computer-based system. It encompasses a system's components, the interactions between them, and the principles guiding its design and evolution.

System Architecture

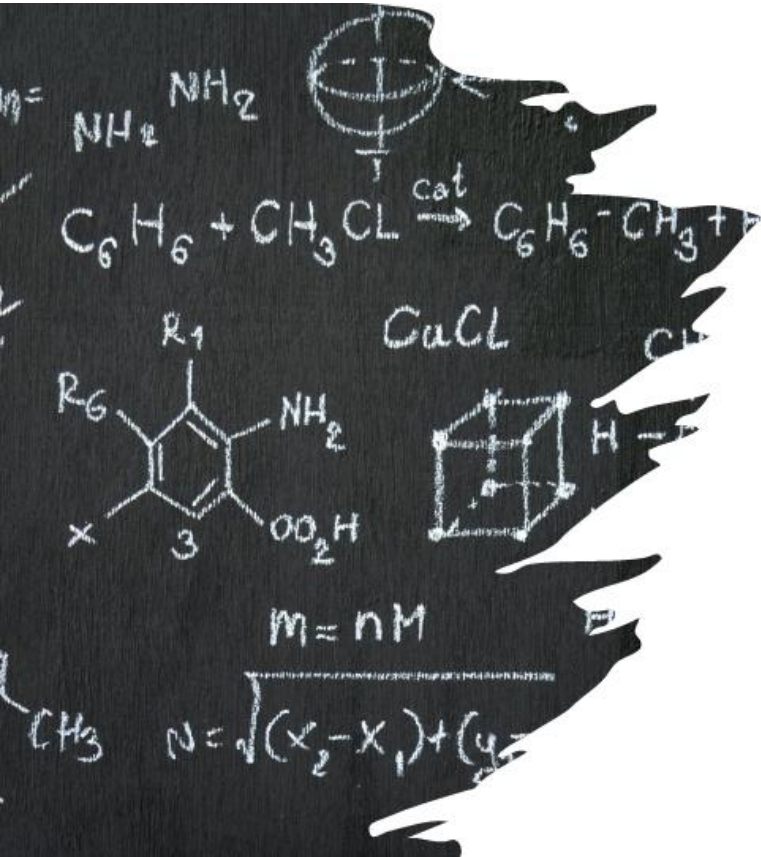
- At a **higher level of abstraction**, an FSM can be viewed as a **process** that consumes inputs and produces outputs.
- A **system** can be described as a set of **communicating processes**.
- Each **process** can be described using **standard programming language constructs**, such as **loops**, **if** and **case** statements, and **assignment** statements.
- Such a **behavioral description** of the system assumes the existence of storage elements for all global and local variables, only.
- It also assumes that **operations**, the reading and storing values, and the transporting of values from storage to functional units and back take no time.
- This model does not include the **concepts of time delay**, **time interval**, or **state**.
- The concept of time is reduced to the order in which statements are executed.

System Architecture

- A typical **process** description of the modulo-3 divider is shown in Figure 12.
- The **loop** can be thought of as a one-state FSM, as in Figure 8.
 - As a one-state FSM, the body of the loop will be computed by the datapath on each clock cycle.
- On the other hand, we can assume that the **conditional expression** in each "if" statement and each branch of each "if" statement are executed in different states.
- Thus, every behavioral description or, in that respect, with every computer program can be trivially implemented with a FSM architecture.
- In reality, each system is described by a set of **processes**, **procedures** and **functions**. Such a system description is partitioned into sub-descriptions, each of which is realized with a system component.

System Architecture

→ **Figure 12:** Behavioral description of modulo-3 divider.



```
Loop forever
  if count=1 then
    if x=2 then
      begin
        x=0
        y=1
      end
    else
      begin
        x=x+1 y=0
      end
    endif
  else
    begin
      x=0
      y=0
    end
  endif
endloop
```

System Architecture

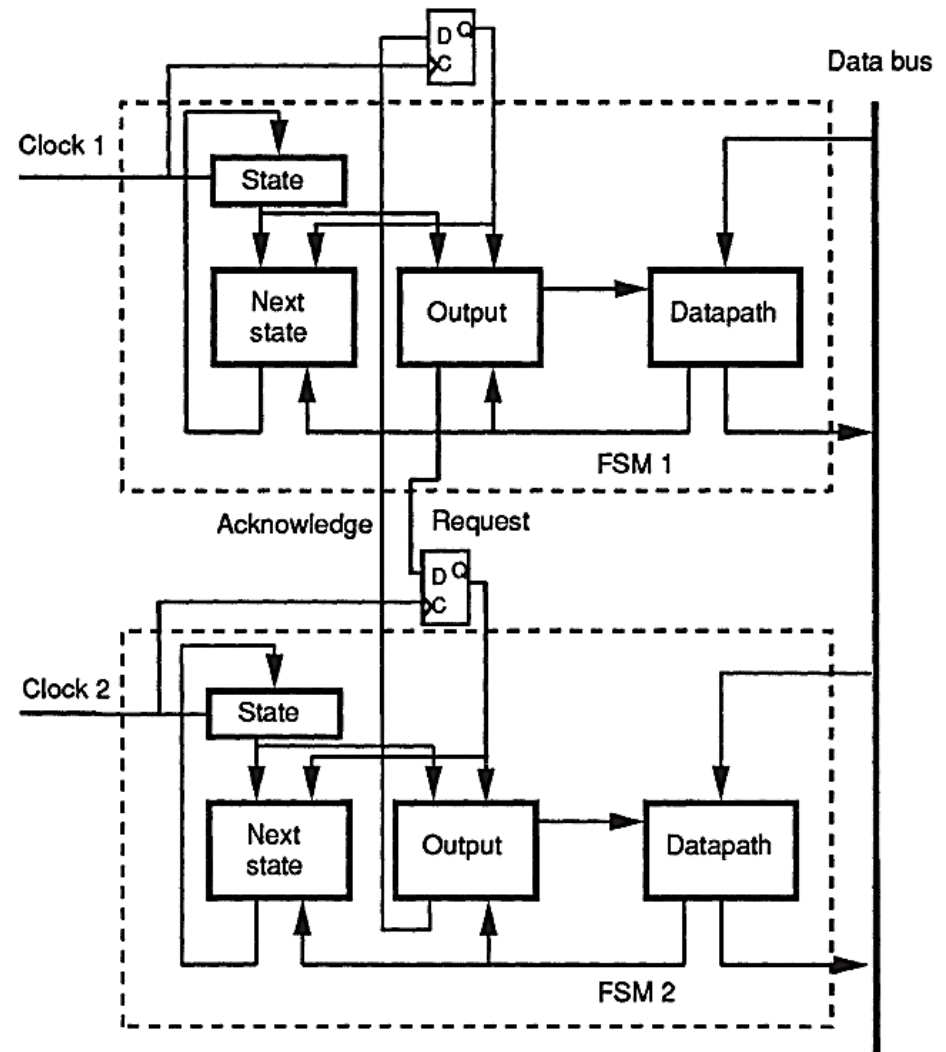
- A system consists of a set of **interconnected components**, such as **processors**, **memories**, **controllers**, **bus arbiters**, **DMA controllers**, and **interface logic**.
- Each component can be implemented with one or more **communicating FSMDs**.
- **Communication** is between **control units**, or between **datapaths**, or between both.
- **Control signals** are rarely used as inputs to the datapaths of other processes, and, likewise, datapath values are rarely used as inputs to other control units.
- The **number of signals** and the **timing relationship** between signals during communication is called a **protocol**.
- If all processes are clocked by the same **clock signal**, the system is called **synchronous**; if clock rates are different, communication is called **asynchronous**.

System Architecture

- In the case of both **synchronous** and **asynchronous** communications, the most frequently used protocol is a **request-acknowledge handshaking protocol**, which is depicted in Figure 13.
- The first FSMD **interrupts** the second FSMD by setting the **request signal** to 1.
- It stays in the request state until the **acknowledge signal** is set to 1 by the second FSMD, at which time it leaves the request state, setting the request signal back to 0.
- The second FSMD jumps to the **acknowledge state** when interrupted. It sets the acknowledge signal to 1 and stays in the acknowledge state until the request is removed, at which time it leaves the acknowledge state.

System Architecture

Figure 13: Communication between FSMs.



System Architecture

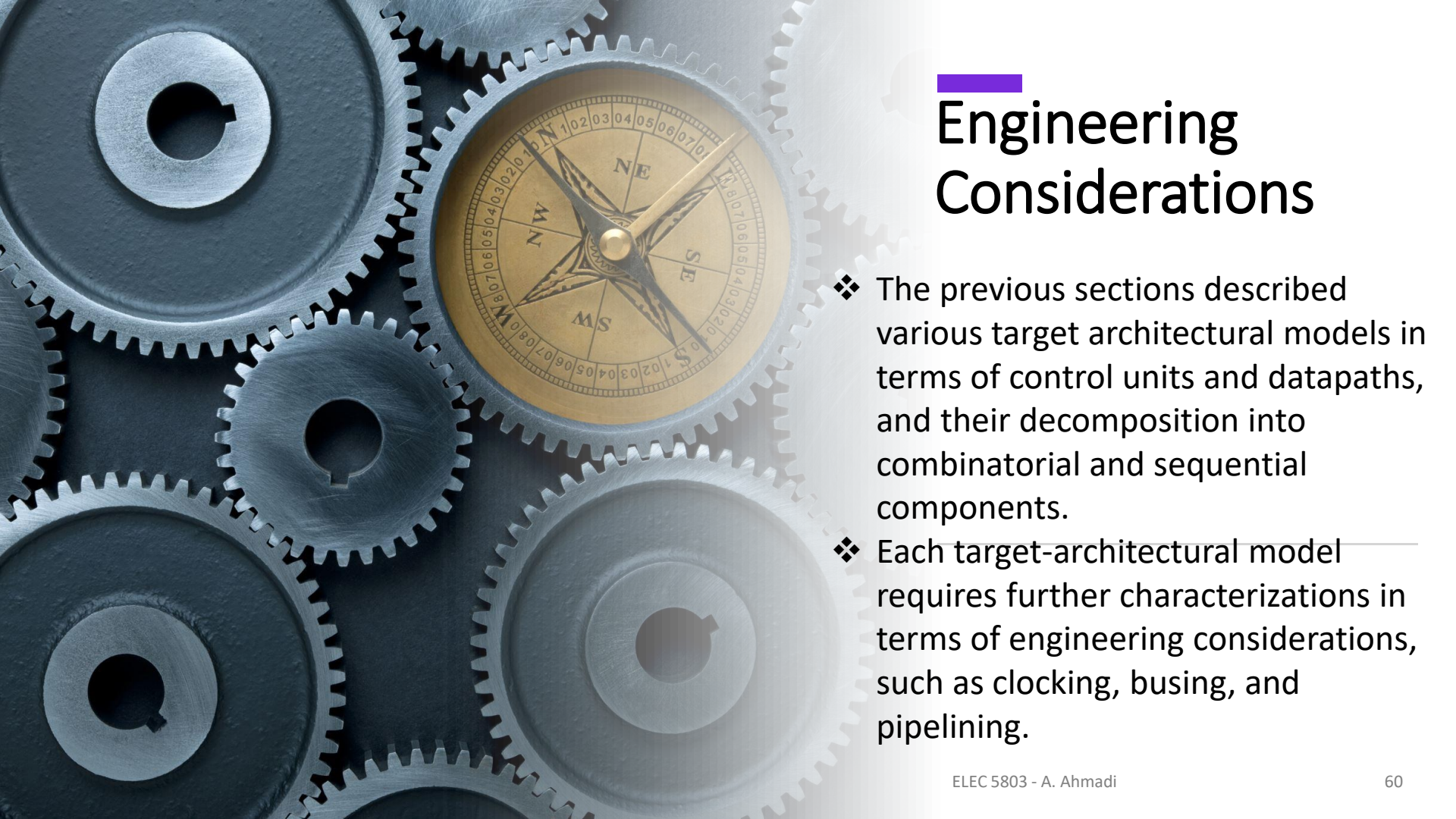
- The **request-acknowledge handshake protocol** can be implemented by two simple D-type flip-flops clocked by the corresponding FSMD **clock signals** shown in Figure 13.
- **Data** can be exchanged between corresponding datapaths in a similar manner.
- The **request-acknowledge sequence** can also be used to free the bus for data transfer.
- After the **acknowledge signal** is received, the first FSMD sets the **data-ready signal** to 1 and puts the data on the bus.
- The second FSMD **reads the data** from the bus and sets the **data-received signal** to 1, after which the first FSMD takes the data off the bus and resets the data-ready signal.
- The second FSMD responds by resetting its **data received signal**.

System Architecture

- The **request-acknowledge handshaking protocol** can be simplified if the **clock rates** are known and the **response time** of each FSMD can be accurately predicted.
- Under these conditions, the first FSMD keeps the data on the bus for a predetermined period of time and does not wait for the data-received signal, since the time needed by the second FSMD to read the data is known.
- Such is normally the case when the second FSMD is a **memory**.
- The design process system synthesis from a behavioral description consists of five steps:
 1. Compilation
 2. Partitioning
 3. Interface Synthesis
 4. Scheduling
 5. FSMD synthesis

System Architecture

- **Compilation** translates the behavioral description, written either in an HDL or a standard programming language, into a standard representation, such as a **control/data flow graph (CDFG)**, that explicitly shows control and data dependences within the input description.
- **Partitioning** divides this representation into **local groups**, i.e., where each group will be implemented on a distinct chip or some other physical medium. The groups are further partitioned into **subgroups**, where each subgroup is implemented using a FSMD architecture; a subgroup can be as large as a processor or as small as a counter.
- Partitioning into chips or between FSMDs requires **well-defined communications**. **Interface synthesis** determines the communication channels and protocols and generates the hardware to execute these protocols.
- **Scheduling** divides the CDFG into states or **control steps**. Variable assignments in the same control step are executed concurrently.
- Such a **scheduled behavioral description** can be then synthesized as an **FSMD**.

The background of the slide features a close-up, artistic photograph of several interlocking metal gears. In the center, a circular compass rose is visible, showing cardinal and intercardinal directions (N, NE, E, SE, S, SW, W, NW) and degree markings. The lighting creates strong highlights and shadows on the metallic surfaces of the gears.

Engineering Considerations

- ❖ The previous sections described various target architectural models in terms of control units and datapaths, and their decomposition into combinatorial and sequential components.
- ❖ Each target-architectural model requires further characterizations in terms of engineering considerations, such as clocking, busing, and pipelining.

Engineering Considerations

- In this section we have very brief review on some engineering aspects of behavioral synthesis such as:
- Clocking system
- Bus (internal data communication) design
- Pipelining

Engineering Considerations: Clocking

- The most basic storage element is a **set-reset (SR) latch**, which can be realized by cross coupling two NAND or NOR gates.
- Such a latch is set into state 1 when the set input is equal to 1 and reset to 0 when the reset input is equal to 1.
- When both inputs are equal to 0, the latch persists in its previous state.
- The case where both inputs are equal to 1 is forbidden because it is difficult to predict which state the latch will enter if both inputs become equal to 0 simultaneously.
- The SR latch is an **asynchronous** element since its output depends only on its inputs.
- Asynchronous circuits that are built from such asynchronous elements are **difficult to debug** because each component latch in the design changes its state at a different time.

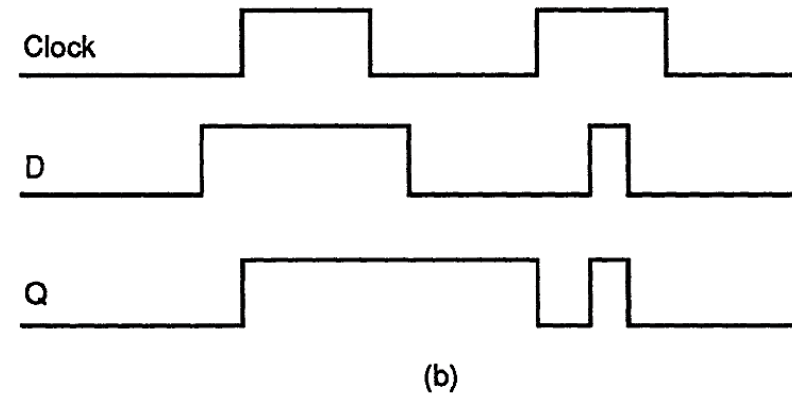
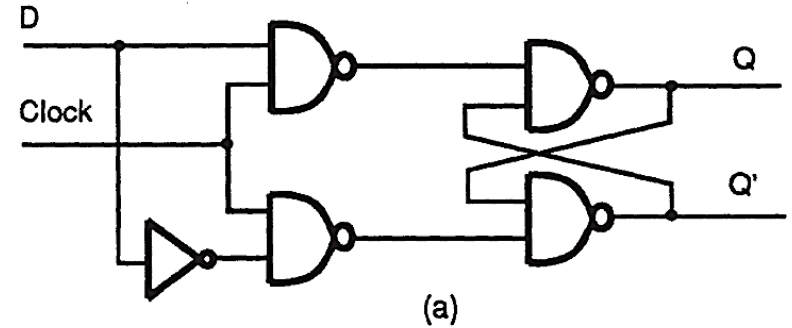
Engineering Considerations: Clocking

- In order to simplify the **debugging**, **testing**, **maintenance**, and **manufacturing** of designs, time is divided into intervals, and all storage elements are set into their respective states at the beginning of each time interval by a clock signal.
- Using a **clock signal**, an SR latch can be converted into the commonly used **D-type latch** shown in Figure 14.
- The behavior of a D-type latch is very simple. The output of the latch follows, with some delay, the input data signal D, when clock = 1. When clock becomes 0, the latch retains the last value of the D input. To prevent any ambiguity, the D input must be stable during a short interval before and after changes in the clock.
- **Clocking** has the advantage of all latches being triggered at the same time. However, there is a risk of **data leakage** from serially connected latches. If the **delay** through a latch is less than the duration the clock signal remains at 1, data can propagate through several latches connected in series, disrupting the orderly transfer of data from latch to latch.

Engineering Considerations: Clocking

Figure 14: D-type latch:

- NAND gate implementation,
- a sample of input and output waveforms.



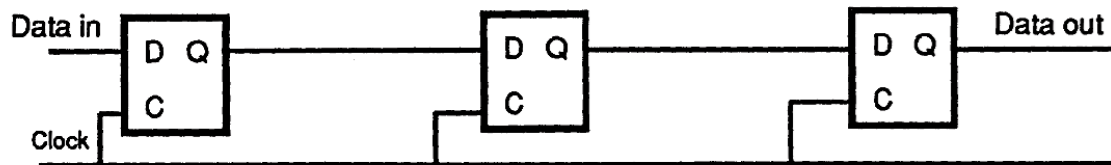
Engineering Considerations: Clocking

- For **example**, consider a simple shift register consisting of three D-type latches, as shown in Figure 15.a.
- If the input-output delay through the latch is t_{DQ} and the clock signal is equal to 1 longer than $3 \times t_{DQ}$, all three latches in the shift register will contain the same input data by the time the clock becomes equal to 0.
- This is obviously erroneous behavior for a shift register. Although we can solve the leakage problem by making clock width $t_w = t_{DQ}$, this solution requires **zero manufacturing tolerances**, which is not possible.
- A more practical solution is to use **master-slave** (MS) flip-flops. A MS flip-flop consists of two latches operating in master-slave mode.
- The master latch is triggered by the clock signal, while the slave latch is triggered by the complement of the clock signal. This ensure that data only propagates through the master latch when Clock = 1 and through the slave latch when Clock = 0, as shown in Figure 15.b.

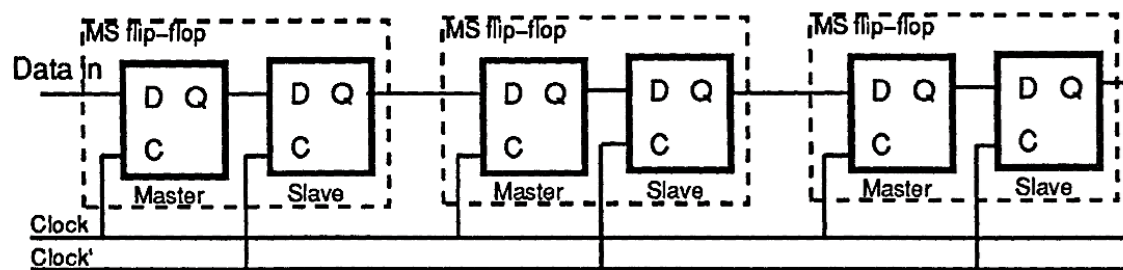
Engineering Considerations: Clocking

→ **Figure 15:** A shift register implemented with:

- a) latches,
- b) master-slave flip-flops.



(a)



(b)

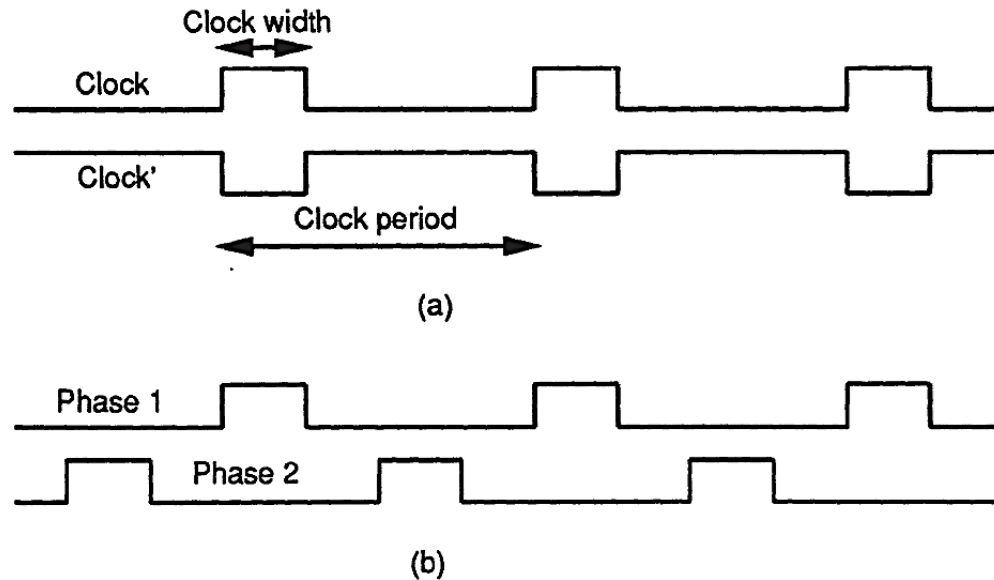
Engineering Considerations: Clocking

- Yet another solution is to use an **edge-triggered** (ET) flip-flop, which can be used equally effectively as a MS flip-flop and at the same cost.
- An ET flip-flop changes its state only during the clock **transition from 0 to 1** (or 1 to 0).
- Since **signal transitions** are not instantaneous, there is still the possibility that both master and slave latches (or two connected ET flip-flops) will be open at the same time.
- To avoid such a hazard, many systems employ a **multi-phase clock**, that is, a set of clock signals that have the same period and are phased in time, such as the 2-phase clock shown in Figure 16.b. One phase serves the role of the clock signal, while the other serves the role of its complement.
- Note that a **clocking strategy** with more than two phases might not add new features to improve the performance beyond the capability of two-phase clocking, although it does add more flexibility to the design phase.

Engineering Considerations: Clocking

→ **Figure 16:** Clocking styles:

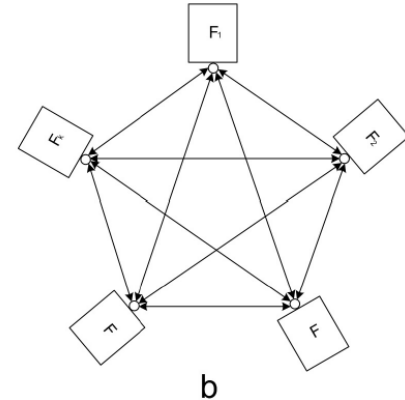
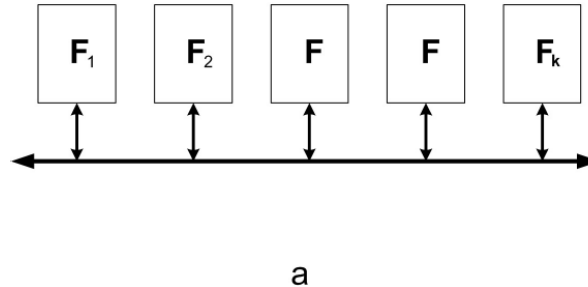
- a) single clock,
- b) 2-phase clock.



Engineering Considerations: Busing

→ System bus structure has a vital impact on the performance because especially in digital systems which deal with an extensive data access, exchange and process. Two extreme cases could be imagined regarding bus interconnections in a system: a **single shared bus** which is accessible for all the sub-blocks without any priority, and a **fully connected peer to peer structure**, This figure shows both cases:

- The first one has a very simple implementation, whereas in massive data processing systems with many sub-blocks is severely slow.
- In the fully connected peer to peer structure, on the other hand, the result system could be faster but more costly in implementation.



Engineering Considerations: Busing

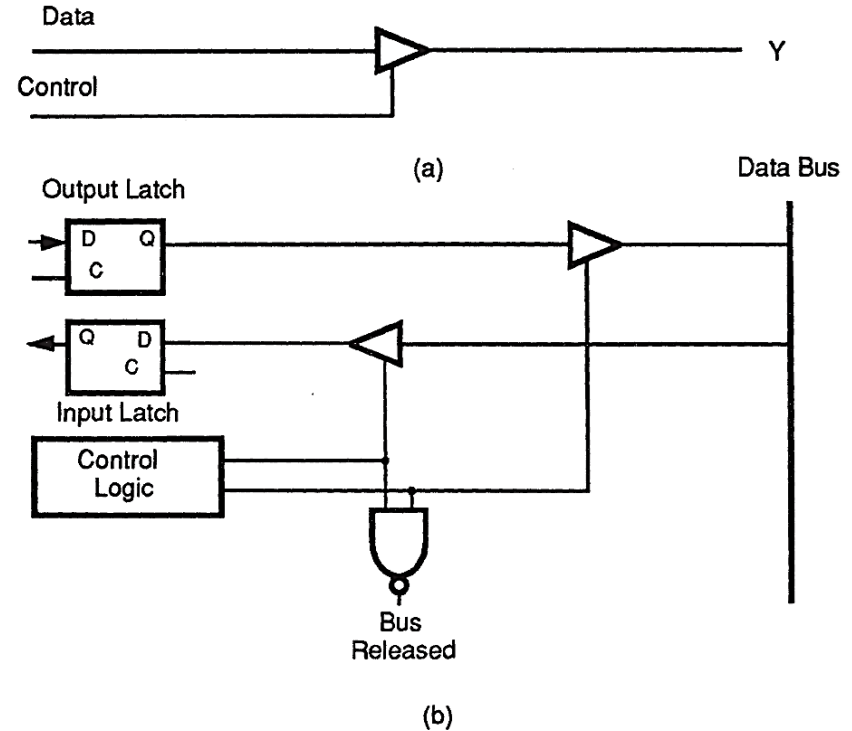
- The most commonly used interconnection unit for chip and system design is a **bus**.
- The popularity of buses can be attributed to their **simple layout** and to the ease with which extra inputs or outputs can be added. The main component in a bus design is a **tri-state driver**, as shown in Figure 17.a, as an **example**. The output Y follows its input data as long as *Control* is equal to 1.
- A bus can be used for communication between different FSMDs in a system. Each FSMD has input and output latches or registers that receive data from a bus or drive the bus using a **tri-state driver**.
- **Tri-state drivers** are controlled by the signals that are generated by the output-function logic of the controlling FSM.
- A **bus controller** is another FSMD. It receives requests for bus use and grants requests using a prioritized or fair-distribution scheme.

Engineering Considerations: Busing

→ **Figure 17:** Bus communication:

- a) tri-state driver symbol,
- b) FSMD interface to a bus.

→ When an FSMD is not driving or receiving data, it signals to the bus controller that it has **released the bus**.



Engineering Considerations: Busing

- The **bus controller** , or **bus arbiter**, communicates with each FSMD through several **control signals**.
- Each bus user usually requests the bus through a **bus-request signal**, and the bus controller grants the bus through a **bus-granted signal**.
- When a bus user does not need the bus anymore, it cancels the request and the bus controller cancels the grant.
- More sophisticated protocols than this one described may be used for controlling the bus.
- Usually the bus controller is a part of one FSMD, such as a processor.
- The **processor** grants the bus to other controllers, such as the direct-memory access (**DMA**) controller, when the processor does not need it for its own use.

Engineering Considerations: Busing

- A **bus arbiter** is a critical component in system design when there are shared bus resources amongst multiple master devices.
- The bus arbiter's role is to **manage access** to the bus and ensure that only one master controls the bus at any given time to avoid conflicts and potential data corruption.
- An overview of the bus arbiter's function:
- **Request and Grant Mechanism:**
 - It listens to requests from multiple masters that want to use the bus.
 - When more than one request is present, the arbiter decides which master gets access to the bus based on predefined arbitration logic or algorithms like round-robin, priority-based, or first-come-first-served.
- **Conflict Resolution:**
 - Conflicts can arise when two or more bus masters simultaneously request access. The arbiter guarantees that only one master has control over the bus during any given cycle to prevent data collisions.
- **Bus Ownership:**
 - The arbiter grants ownership of the bus to one master at a time by sending a grant signal to the selected master. The granted master can then perform its data transfer unhindered.

Engineering Considerations: Busing

→ Fairness and Latency:

- In systems where several masters frequently contend for bus access, the arbiter ensures fairness so that no master is starved of bus access.
- Depending on the arbitration scheme, the arbiter also plays a role in determining the latency of the bus access for each master.

→ Maintaining System Performance:

- By efficiently managing bus requests, the arbiter helps in maintaining overall system performance and preventing bottlenecks in data transfer.

→ Scalability:

- The design of the arbiter can impact the scalability of the bus system. A well-designed arbiter can accommodate the addition of new bus masters with minimal modification.

→ Integration:

- The bus arbiter can be integrated into the bus interface logic or exist as a separate entity in the system.

Engineering Considerations: Pipelining

- Each **functional unit** takes some nominal amount of time, called **unit delay**, to compute a result from the values of its operands.
 - If the **clock period is long** and **unit delays are short**, it is possible to **chain functional units** and perform two or more operations on the same data in a single clock cycle.
 - If the a **unit's delay is long** compared to the **clock cycle**, it may take **several clock cycles** to obtain a result. A control unit must account for such **multicycle delays**, so that the result from a multicycle unit is not needed for as many clock cycles as it takes to generate that result.
- In either case, the functional units are **not highly utilized**!
- In the case of **chaining**, each chained unit is used only during a fraction of the cycle and is idle otherwise.
- Similarly, in the case of **multi cycling**, the entire unit is blocked during several clock cycles, even though the result is computed only in a small portion of the unit.

Engineering Considerations: Pipelining

- **Pipelining** increases the utilization of functional units.
- Each unit is **divided into stages** and the **partial results** are **latched** after each stage.
- The clock cycle can be shortened to equal the delay of the longest stage.
- This method allows **concurrent operation** on several pairs of operands, each getting partially computed in one stage of the pipelined unit.
- The total operation delay may even increase slightly because of the extra time needed to latch partial results, but the total throughput on an independent stream of operands is proportional to the number of stages.

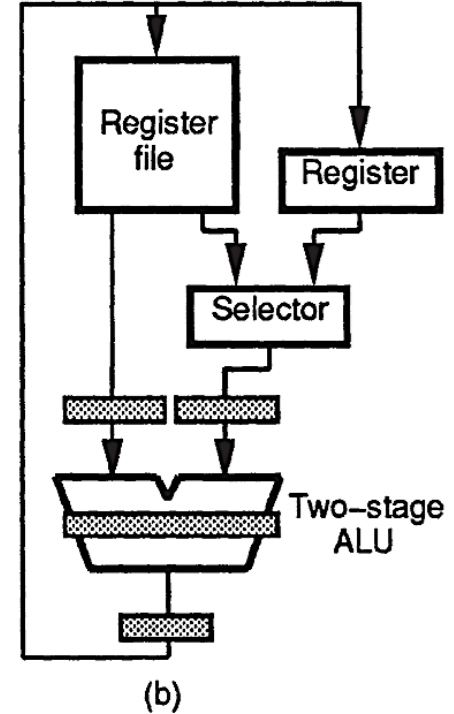
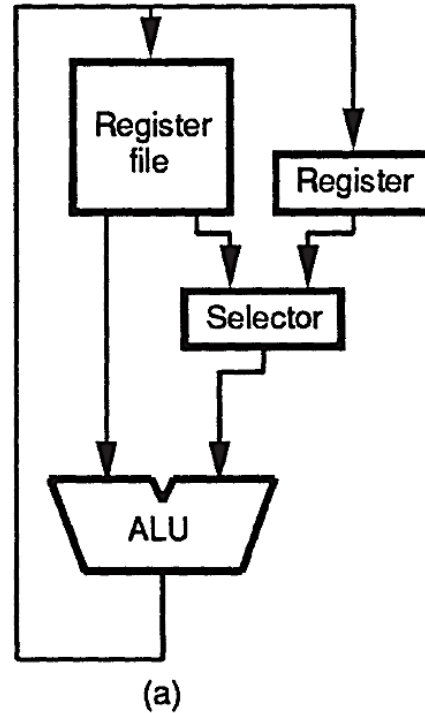
Engineering Considerations: Pipelining

- For **example**, Figure 18 illustrates the difference between nonpipelined and pipelined datapaths.
- The ALU is divided into two stages, but the entire register-to-register transfer is divided into four stages by providing input and output latches for the ALU.
- Thus, any value can be read out of the register file only four clock cycles after its computation is started.
- Similarly, a **control unit** can be pipelined by dividing its next-state and output logic into stages and inserting latches between each stage.
- Since the control logic does not have many levels of gating, designers usually insert latches only at the input and output of the control logic, as shown in Figure 19.

Engineering Considerations: Pipelining

→ **Figure 18:** Datapath pipelining:

- a) nonpipelined datapath,
- b) pipelined datapath with 2-stage adder.



Engineering Considerations: Pipelining

→ **Figure 19:** Control unit pipelining:

- a) nonpipelined control unit,
- b) pipelined control unit.

